

Day 29: Power Management & Compiler Optimization

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Zephyr PM Subsystem

Zephyr's Power Management (PM) subsystem (`CONFIG_PM=y`) automatically transitions the system to the lowest available sleep state during idle periods:

- When no thread is ready to run, the idle thread calls `pm_system_suspend()`
- The PM system selects the deepest sleep state compatible with the next scheduled wake-up
- The kernel resumes from the sleep state when the next tick or IRQ arrives

pm_device API

Individual peripherals can be suspended when not in use to save power:

```
// Suspend a peripheral (reduces power draw)
pm_device_action_run(uart_dev, PM_DEVICE_ACTION_SUSPEND);

// Resume when needed
pm_device_action_run(uart_dev, PM_DEVICE_ACTION_RESUME);
```

Not all peripherals support PM — check `pm_device_action_run()` return value.

Sleep States on nRF52840 (via Zephyr PM)

Zephyr State	nRF52840	Current draw
<code>PM_STATE_RUNTIME_IDLE</code>	WFE/WFI	~1.5 mA

Zephyr State	nRF52840	Current draw
<code>PM_STATE_SUSPEND_TO_IDLE</code>	System ON, RAM retained	~2 μ A
<code>PM_STATE_STANDBY</code>	System ON, partial RAM	~1.5 μ A
(not via Zephyr PM)	System OFF	~0.4 μ A

Enable with: `CONFIG_PM=y`, configure latencies in DTS `power-states` node.

CONFIG_PM_DEVICE

Enable per-device power management: `CONFIG_PM_DEVICE=y`. This allows drivers to implement suspend/resume callbacks that Zephyr calls automatically when devices are not in use.

Compiler Flags: -Os, LTO, gc-sections

- `-Os` (size optimization): reduces code size at the cost of some speed — ideal for embedded
- `-flto` (Link Time Optimization): removes dead code across compilation units, inlines across files
- `-ffunction-sections -fdata-sections`: places each function/variable in its own ELF section
- `--gc-sections` (linker): removes sections not referenced by any other section (dead code elimination)

In Zephyr: `CONFIG_SIZE_OPTIMIZATIONS=y` enables `-Os`. LTO: `CONFIG_LTO=y` (experimental).

Measuring Power with Nordic PPK2

The **Nordic Power Profiler Kit 2 (PPK2)** is a low-cost current measurement tool that works with nRF52840DK:

1. Connect PPK2 between USB and nRF52840DK power supply
2. Use nRF Power Profiler app (Nordic) to measure current vs time
3. Correlate sleep/wake events with GPIO toggle (measured on PPK2 logic input)

Low-Power UART Patterns

UART prevents deep sleep because the receiver must stay active. Strategies:

- Use **UART with RTS/CTS** — host deasserts RTS when no data pending
 - Use **wake-on-UART** (nRF52840 UARTE feature): configure UARTE to wake from deep sleep on RX line activity
 - Replace UART with **BLE** or **NFC** for battery-powered devices
-

Practice Tasks

1. Add `CONFIG_PM=y` and `CONFIG_PM_STATS=y` — use `pm_dump_stateinfo()` to see time spent in each sleep state
 2. Measure current draw with and without `CONFIG_PM=y` — observe the difference with PPK2 or a multimeter
 3. Build your Day 7 example with `-Os` and without — compare binary sizes from `west build` output
 4. Implement wake-on-UART: deep sleep until a byte is received on UART RX — measure wake latency
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day29
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day29/main.cpp](#)